



*** Republic of Tunisia ***

Minister of Higher Education, Scientific Research,
and Information and Communication Technology
General Directorate of Technological Studies
Higher Institute of Technological Studies of Beja
Department of Information Technology

Graduation Project Dissertation

to obtain the National Diploma of Applied License in Development of Embedded and
Mobile Systems

Dev and Deployment of an automated real-time monitoring system for TESCA group

Carried out by :

Fedi Amdouni & Firas Ouesleti

Defended on 00/00/2026, in front of the committee members :

- | | |
|-------------------------|-----------------------|
| • Mrs. Balkiss Bettoumi | President |
| • Mr . Hattab Ayari | Reviewer |
| • Mr . Radhouane Aloui | University supervisor |
| • Mr . Fethi Yahyaoui | Company supervisor |

End of studies internship completed at

Host company



Client company

tesca

Academic year 2025-2026

General Directorate of Technological Studies
Higher Institute of Technological Studies of Beja
Department of Information Technology

Graduation Project in Development of Embedded and Mobile Systems	
Made by: Fedi Amdouni Firas Ouesleti	Batch: 2025-2026
Title: Dev and Deployment of an automated real-time monitoring system	
Defence: 18/06/2026	
Summary: In modern manufacturing, accurate production monitoring is essential for optimizing efficiency. However, many facilities face unreliable traceability of machine stoppages and their root causes, hindering precise Overall Equipment Effectiveness (OEE) calculation and performance analysis. This project addresses this challenge by designing and deploying an automated, real-time industrial monitoring system. Built on a multi-layer architecture. The system automatically detects stoppages, enables real-time cause logging, and centralizes production data in a MySQL database, while analytical dashboards provide dynamic KPI visualization. Implementation results demonstrate complete machine event traceability, automatic OEE calculation per shift and day, and a significant reduction in manual entry errors and undetected production losses. The system successfully enhances production transparency and decision-making.	
Keywords: Industrial monitoring, OEE, Modbus TCP, PLC, Node-RED, NestJS, HMI, machine stoppages, MySQL database	
Résumé: Dans l'industrie moderne, une surveillance précise de la production est essentielle pour optimiser l'efficacité. Cependant, de nombreuses installations font face à une traçabilité peu fiable des arrêts machine et de leurs causes profondes, entravant le calcul précis de l'Efficacité Globale des Équipements (OEE) et l'analyse des performances. Ce projet relève ce défi en concevant et déployant un système de surveillance industrielle automatisé et en temps réel. Construite sur une architecture multi-couches, . Le système détecte automatiquement les arrêts, permet l'enregistrement en temps réel des causes, et centralise les données de production dans une base de données MySQL, tandis que des tableaux de bord analytiques fournissent une visualisation dynamique des indicateurs de performance (KPI). Les résultats de l'implémentation démontrent une traçabilité complète des événements machine, un calcul automatique de l'OEE par équipe et par jour, et une réduction significative des erreurs de saisie manuelle et des pertes de production non détectées. Le système améliore avec succès la transparence de la production et la prise de décision, avec des travaux futurs prévus pour l'extension à d'autres lignes de production.	
Mots-clés: Surveillance industrielle, OEE, Modbus TCP, PLC, Node-RED, NestJS, IHM, arrêts machine, base de données MySQL	

Dedication

First and foremost, I express my deepest gratitude to Allah for His infinite blessings, wisdom, and strength throughout this journey. His guidance has been my compass in overcoming challenges and achieving this milestone.

To my favorite cheerleaders in life,

To my parents,

The architects of my dreams and the guardians of my laughter. Your love has painted my world with vibrant hues of happiness and warmth. This piece is just a little way to show how much I appreciate all the sacrifices you've made and the endless love you've given me.

To my brother,

The partner in crime, with your playful banter and unwavering support, you've made every moment an adventure worth living. This dedication is a reminder of the special bond we share and the joy you bring into my life.

To my dear friends,

The stars that light up my sky on the darkest of nights. Your friendship is my greatest treasure. This work is dedicated to you as a symbol of our enduring bond and the countless memories we've yet to create.

May this dedication serve as a humble token of my gratitude to all those who have touched my life with their kindness, wisdom and love.

Fedi Amdouni

Dedication

First and foremost, I would like to thank God for all His blessings upon me and my family. For the strength, patience, and courage He granted me each day to pursue my goals and accomplish this final year project.

I dedicate this humble work to:

To my beloved parents,

Your unwavering support, endless sacrifices, and constant prayers have been the foundation of my success. May Allah grant you long life, good health, and the happiness you truly deserve.

To my dear sister,

For your encouragement, understanding, and the joy you bring to my life. Your belief in me has been a constant source of motivation.

To my partner, Fedi Amdouni,

For your dedication, teamwork, and perseverance throughout this project. This achievement is as much yours as it is mine.

To all my friends and mentors,

Who have guided, supported, and inspired me along the way. Your wisdom and kindness have shaped my path.

To everyone who contributed to this journey, directly or indirectly, I offer my sincere gratitude.

Firas Ouesleti

Acknowledgments

The work presented in this report was carried out at the Higher Institute of Technological Studies of Beja (ISET Beja) in collaboration with the TESCA Group. As we conclude this pivotal chapter of our academic journey, we wish to express our deepest gratitude to all those who contributed to the successful completion of this project.

Foremost, we extend our sincere appreciation to **Mr. Radhouane Aloui**, our university supervisor. His expert guidance, unwavering support and insightful mentorship have been instrumental throughout this endeavor. His constant feedback and encouragement pushed us to exceed our limits, particularly in exploring advanced Automation and embedded systems concepts, and we are truly grateful for his dedication to our academic and professional growth.

We are equally indebted to our industry supervisor, **Mr. Fethi Yahyaoui**, for his invaluable presence and hands-on guidance during our internship at TESCA Group. His willingness to explain complex industrial processes, challenge our critical thinking and bridge the gap between theoretical knowledge and real-world application has been profoundly inspiring. We deeply appreciate his commitment to fostering our development both as students and as professionals.

Our gratitude also extends to the entire team at TESCA Group for providing us with a stimulating, collaborative and supportive work environment. The opportunity to apply our skills in a dynamic industrial setting has been an invaluable learning experience that will undoubtedly shape our future careers.

We would like to thank the faculty and staff of ISET Beja for the strong educational foundation they have provided us over the past three years. The knowledge, technical skills and professional values we have acquired within this institution have been the cornerstone of this project.

We sincerely thank the members of the jury for taking the time to evaluate this work. We deeply appreciate your efforts and look forward to your constructive feedback and insights.

Finally, we extend our heartfelt thanks to everyone who supported us directly or indirectly throughout this journey. Your encouragement, patience and belief in our potential have been our greatest motivation. This achievement is as much yours as it is ours.

Fedi Amdouni & Firas Ouesleti

Contents

Acknowledgments	v
General Introduction	1
1 Preliminary analysis	2
1 Presentation of the enterprises	2
1.1 TS2I	2
1.2 TESCA	3
1.2.1 Presentation of the company	3
1.2.2 Technical file	5
1.2.3 Organizational chart	5
2 Case study	6
2.1 Description of the existing system	6
2.1.1 Overview of the Existing System	6
2.2 Criticism of the existing system	6
2.2.1 Limited Data Analysis	6
2.2.2 No Real-Time Manager Updates	6
2.2.3 Limited Performance Metrics	7
2.2.4 Inflexible User Interface and Limited Access	7
2.2.5 Data Quality and Verification	7
2.2.6 Lack of Security and Integration	7
2.2.7 Scalability Concerns	7
3 Modeling languages: SysML	7
2 Architecture and Design	9
3 Implementation	10
4 SQL Optimisation	11
1 Context and System Overview	11
1.1 Context	11
1.2 System Overview	11
1.2.1 Database Structure	11
1.2.2 Production Context	12
1.2.3 The Key Metric: Percentage of Daily Downtime (PCT)	12
2 Query A: The Original Implementation	12
2.1 Implementation	12

2.2	Initial Behavior	14
2.3	The Root Cause: A Fully Correlated PCT Sub-query	14
2.4	SHOW PROFILE Analysis	14
3	Query B: Optimized Implementation	15
3.1	Motivation	15
3.2	Experimental Setup	15
3.3	Experiment B1: Simple Indexes	16
3.3.1	Hypothesis	16
3.3.2	Results	16
3.4	Experiment B2: Functional Index	17
3.4.1	Hypothesis	17
3.4.2	Results	17
3.5	Experiment B3: Covering Index	18
3.5.1	Hypothesis	18
3.5.2	Results	18
3.5.3	EXPLAIN ANALYZE Output	19
3.6	Side by Side Comparison	20
4	Query C: Breaking the Quadratic Loop	21
4.1	The Targeted Fix	21
4.2	Results	22
4.3	EXPLAIN ANALYZE: Diagnosing the Remaining Bottlenecks	22
5	Query D: The Window Function Hypothesis	23
5.1	The Reasoning Behind the Attempt	23
5.2	Implementation	24
5.3	Measured Result	24
5.4	Root Cause: Sort Buffer Spill to Disk	25
5.4.1	How MySQL Executes Window Functions	25
5.5	Learnings from Query D	25
6	Query E: Big Data Principles Applied to MySQL	25
6.1	Schema Redesign	26
6.1.1	Stored Generated Columns	26
6.1.2	Why Each Index Exists	26
6.2	The Optimized Query	27
6.2.1	Results	28
7	Performance Visualization	29
7.1	Query Execution Time Comparison	29
A	EXPLAIN ANALYZE Traces	32
1	Query B on stopsB1 (LIMIT 50)	32
B	Optimized Table Schema with Stored Generated Columns	33
1	Complete Schema Definition	33

List of Figures

1	Tesca’s partners	3
2	Organizational chart of TESCA	5
1	Query performance across row counts (logarithmic scale)	29

List of Tables

1	Technical file of TESCA	5
2	Core stops table structure	12
3	Query A measurements	14
4	SHOW PROFILE results for Query B	15
5	Index configuration summary for all three experimental tables	15
6	stopsB1 benchmark results with simple indexes	16
7	stopsB2 benchmark results with functional indexes	17
8	stopsB3 benchmark results with covering indexes	19
9	Index strategy comparison: Query B on stops table	20
10	Query C performance compared to Query B3	22
11	EXPLAIN ANALYZE profile for Query C at 1M rows	23
12	Query D benchmark results on production dataset	24
13	Query C vs Query D — what actually happens at 1M rows	25
14	Index inventory and purpose	26
15	EXPLAIN ANALYZE profile for Query E at 1M rows	28

General Introduction

In today's competitive industrial landscape, production efficiency is no longer a luxury, it is a necessity. For manufacturing firms, production efficiency is essential because their production process faces numerous challenges, and they must be ready at all times to optimize production output, reduce wastage, and mitigate any disturbances that may arise within the production floor. One of the critical indicators of the effectiveness of production processes is the Overall Equipment Effectiveness (OEE), known in French as the *Taux de Rendement Synthétique* (TRS), stands out as one of the most comprehensive metrics, capturing the combined impact of availability, performance, and quality losses in the production process. [1].

The automotive industry, governed by rigorous quality standards such as IATF 16949 [2], is particularly demanding in this regard. Tunisia has established itself over the decades as a competitive industrial hub for automotive suppliers, attracting major international players thanks to its geographic proximity to Europe, its cost-efficient production and skilled workforce [3]. Among these companies is TESCA Group, an international manufacturer specializing in vehicle interior components, with several production sites operating across Tunisia [4].

TESCA Group has always prioritized continuous improvement. Better visibility into production matters—detecting stoppages quickly, understanding what causes them, and responding fast directly impact competitiveness. To achieve this, they developed a production monitoring application that automates data collection and OEE calculation.

But operational use revealed real problems with the Node-RED setup. The dashboard can't be customized, visualizations are limited, and it doesn't scale. Integration is restricted. The company also has no mechanical or electrical documentation, which means maintenance teams can't diagnose equipment problems or plan maintenance work properly. These gaps stop the system from supporting TESCA Group's improvement goals.

Our Final Year Project tackled these issues in three ways. We built a modern dashboard to replace the old interface—one that's easier to use and lets teams customize what they see. We rewrote the backend from scratch on a proper server, which means better performance and reliability. We also put together complete documentation for the mechanical and electrical systems so maintenance crews can actually fix things without guessing. The result is a monitoring system that gives TESCA Group real visibility into what's happening on the floor, helps them spot problems faster, and makes their continuous improvement work actually possible.

This report summarizes all the work completed during this Final Year Project. It documents five key areas of development: first, preliminary analysis presenting the two hosting organizations and covering the project context and the limitations of the existing Node-RED implementation. Second, the development of a modern, dedicated frontend dashboard with professional data visualization and customizable layouts. Third, the backend architecture migration and automation programming, and HMI logic development. Fourth, the database optimization to improve system performance, reliability, and scalability. And finally, the creation of comprehensive maintenance plans and technical documentation for mechanical and electrical systems to support long-term operations.

Chapter 1

Preliminary analysis

Introduction

The first chapter of this report aims to provide an overview of the host and the client companies, present the case study by highlighting the problem to be addressed, and introduce the modeling language used for the project. This chapter sets the stage for the subsequent sections by establishing the context, significance, and scope of the actions undertaken in the next chapters.

1 Presentation of the enterprises

1.1 TS2I

TS2I is a leading company in the field of specialised machines fabrication, automation, industrial maintenance, and Tampographique service. With a strong commitment to quality, innovation, and customer satisfaction, TS2I has established itself as a trusted partner for various industries. The following provides an overview of TS2I and highlights its expertise in fabrication machines, automation, industrial maintenance, and pad printing services.

- **Fabrication of specialised machines:** TS2I specializes in the design, development, and production of fabrication of specialised machines. These machines are tailored to meet the unique requirements of clients in diverse industries such as automotive, aerospace, electronics, and more. With a team of experienced engineers and technicians, TS2I leverages cutting-edge technologies to deliver custom fabrication machines that optimize productivity, efficiency, and quality. Whether it's automated assembly lines, robotic systems, or specialized machining equipment, TS2I ensures that each machine is built to the highest standards.
- **Automation:** Automation plays a crucial role in enhancing productivity and reducing operational costs. TS2I offers comprehensive automation solutions to help businesses automate their processes. From simple control systems to complex integrated automation solutions, TS2I designs and implements tailored automation solutions based on clients' requirements. Their expertise in PLC programming, HMI design, robotics, and motion

control allows them to deliver efficient and reliable automation solutions that streamline operations and improve overall efficiency.

- **Industrial maintenance:** TS2I understands the importance of maintaining industrial equipment to ensure optimal performance and minimize downtime. The company offers comprehensive maintenance services to address the unique needs of its clients. Whether it's preventive maintenance, troubleshooting, or equipment repair, TS2I's skilled technicians and engineers are well-equipped to handle a wide range of industrial machinery. By providing regular maintenance, TS2I helps clients avoid costly breakdowns, increase equipment lifespan, and maximize productivity.
- **Tampographique service:** In addition to its fabrication machines and automation expertise, TS2I also offers a specialized service in tampography, or pad printing. Tampography is a versatile printing technique used for applying designs or logos onto various surfaces, including plastics, metals, ceramics, and glass. TS2I's pad printing service ensures high-quality and precise printing, providing clients with branding solutions that are durable and visually appealing. Whether it's product customization, promotional items, or industrial labeling, TS2I's pad printing service offers reliable and efficient solutions.

1.2 TESCA

1.2.1 Presentation of the company

Tesca Group is a family-run company with a background in textiles that has successfully ventured into the automotive industry, specifically automotive seating. Emulating the approach of a French automotive manufacturing house, Tesca Group meticulously crafts each piece, from its atelier to its global production sites[4]. The company strives to embody the same creative audacity and prides itself on being a partner to major automotive manufacturers like:



Figure 1: Tesca's partners

With a clientele consisting of demanding visionaries who shape the future of mobility, Tesca Group positions itself as a reliable and flexible partner, accompanying its clients in their pursuit of innovation and competitiveness. The company believes that adopting a long-term and eco-friendly vision is crucial for sustainable growth, constantly pushing for excellence rather than settling for mediocrity[4].

-
- **Sustainability:** A concrete and integral part of Tesca Group’s approach. The company is committed to sustainable development and actively incorporates environmental and social values into its products, services, and activities within the automotive industry. Since 2004, Tesca Group has been dedicated to controlling its ecological footprint, and the executive management maintains and expands upon these efforts. The company adheres to the values and principles of the Global Compact through its Ethics Charter, to obtain ISO 14001 certification for all its sites[4].
 - **Environmental policy:** Tesca Group’s environmental policy revolves around five major directions: behaving in an environmentally responsible manner, optimizing and controlling the industrial waste, optimizing energy consumption and natural resources, participating in the reduction of the ecological footprint throughout the design and production processes, and ensuring compliance with environmental regulations and requirements[4].
 - **Innovation:** A driving force within Tesca Group, with a constant emphasis on generating new ideas and executing them effectively. The company’s research programs focus on key themes in the automotive industry, such as weight reduction, optimized part design, and ecological footprint expertise, while prioritizing user comfort, functionality, and safety. Tesca Group holds numerous patents in the conception, design, and manufacturing of automotive parts and seat components, including headrests, armrests, upholstery, padding, and “smart textiles”[4].
 - **Industrial excellence:** A cornerstone of Tesca Group’s strategy, whether through its local production sites or research centers. The company strives to provide reliable processes that meet the evolving market expectations worldwide. Tesca Group empowers its teams through a culture of continuous improvement, with competitiveness, efficiency, and exceptional service forming the foundation of its industrial success. SPRINT (Tesca’s Industrial Production System) is employed to engage collaborators at all levels, aiming to standardize procedures and ensure perpetual progress in industrial performance. Shared values among Tesca Group employees include excellence, self-discipline, commitment, autonomy, and pragmatism[4].
 - **Quality:** Tesca Group’s Quality policy aligns with five major directions: meeting commitments to conformity, cost, and deadlines; aligning industrial and purchasing performance with the highest quality criteria; standardizing existing processes to offer innovative products; prioritizing process control and prevention; and continuously enhancing the skills of its teams[4].
 - **History:** In terms of its history, Tesca Group has a notable timeline of achievements. Starting in 1960 with the provision of Citroën 2CV roofs, the company went on to pioneer and apply the Foam In Place technology in 1978. In 1983, Tesca Group established its Design Studio in Paris, followed by the creation of the CERA research and development center in Reims in 1993. By the year 2000, the company had expanded its production capabilities worldwide. In 2016, Tesca Group underwent a significant transformation, becoming Trèves TSC[4].
-

Overall, Tesca Group has established itself as a reputable partner to major automotive manufacturers, driven by a commitment to sustainability, innovation, industrial excellence, and quality. Through its diverse range of products and services, the company aims to shape the future[4].

1.2.2 Technical file

The following table presents the technical file of TESCA.

Table 1: Technical file of TESCA

Company name	TESCA
Sector	Vehicle interior equipment
Creation	2019
Legal form	Foreign company
Staff	179 employees
Address	Grombalia Industrial Zone – Nabeul– Tunisia

1.2.3 Organizational chart

The organizational chart shown in Figure 2 presents the distribution of positions and functions within the Tesca enterprise.

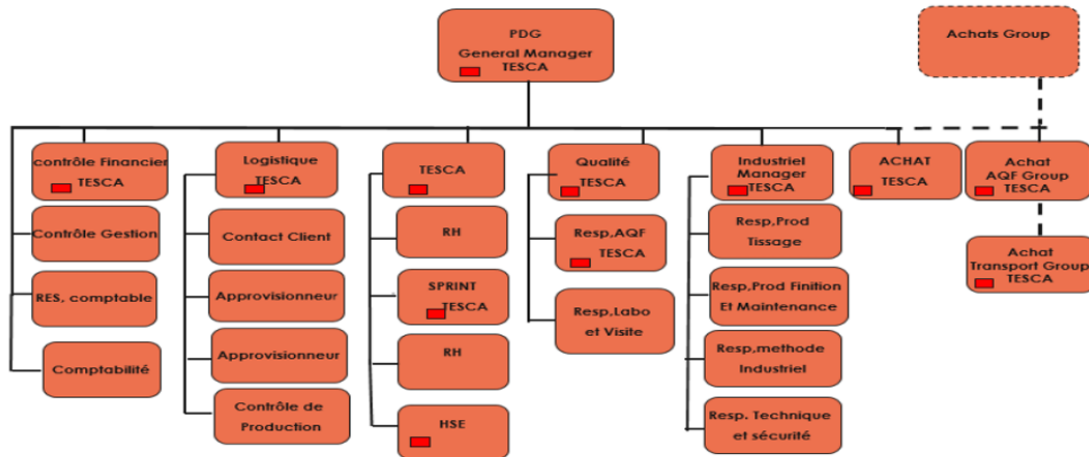


Figure 2: Organizational chart of TESCA

2 Case study

2.1 Description of the existing system

2.1.1 Overview of the Existing System

The existing production monitoring system was developed in the previous internship project to replace manual paper-based stop recording with automated digital tracking. The system consists of three components working together: a Delta DVP PLC installed on the fabric inspection machine, a Node-RED platform serving as the intermediary that reads PLC data and communicates with the database, and a MySQL database storing all production information.

Before this system existed, operators manually recorded all machine stops, causes, and durations on paper during their shifts. This manual approach made monitoring difficult, introduced frequent errors, and prevented real-time analysis. The new system automates data collection: when the machine stops, the PLC detects it, the operator selects the cause through a tablet interface, and the system automatically calculates the duration when the machine restarts.

The system serves two main user groups. Operators on the production floor use a tablet application that displays real-time machine speed, yardage measurements, lighting controls, and a dropdown menu to select stop causes. This interface updates continuously and provides immediate feedback, making it practical for shop-floor use. Managers access a web interface where they can filter data by date and shift team, view a table of all stops with their causes and durations, see a bar chart showing which causes are most frequent, and view the daily OEE (Overall Equipment Effectiveness) metric that measures productive time versus downtime.

2.2 Criticism of the existing system

2.2.1 Limited Data Analysis

The existing system successfully records stops but provides limited analytical capability. The manager interface shows a bar chart of stop causes and a table of stops for the selected period, but it does not show trends over time. A manager viewing one week of data sees aggregated numbers but cannot easily see whether the situation is improving or deteriorating day by day. The interface does not enable comparison between teams or shifts, making it difficult to identify performance patterns.

2.2.2 No Real-Time Manager Updates

A significant limitation is that the manager web interface does not automatically update. When a manager opens the dashboard to check production status, they must manually click a filter button to retrieve current data. If new stops occur after they view the data, these are not automatically displayed—the manager sees the situation as it was at their last manual refresh. In production environments, delays in visibility can mean slower response to problems.

2.2.3 Limited Performance Metrics

The system primarily reports OEE and stop causes. Modern production monitoring typically tracks additional metrics to provide a complete picture. The system does not show trends in OEE over time, separate availability from performance metrics, or provide any predictive capability to identify potential problems before they occur. A manager cannot easily identify whether equipment health is improving or deteriorating, or whether specific causes show patterns that might indicate underlying issues.

2.2.4 Inflexible User Interface and Limited Access

All managers see the same view regardless of their role or needs. The interface is designed for desktop computers and is not optimized for mobile devices, meaning managers cannot check production status from a smartphone while away from their office. There is also no concept of custom dashboards where different users might want different information.

2.2.5 Data Quality and Verification

The system relies on operators to manually select stop causes from a dropdown menu. There is potential for human error or inconsistency in cause selection. Once a cause is recorded, there is no verification process or audit trail to show who entered the data or when. If a data discrepancy is discovered later, there is no way to trace back what happened.

2.2.6 Lack of Security and Integration

The existing system has no user authentication or access control. Anyone with the IP address can access all production data without logging in. The system also cannot export data to other formats or systems. If a manager needs to perform advanced analysis in Excel or if another department needs production data, it must be manually copied or requested.

2.2.7 Scalability Concerns

The Node-RED platform handles all functions in a monolithic design: reading from the PLC, storing in the database, and serving both the operator tablet and manager web interface. As data volume grows or additional machines are added, this architecture may face performance or reliability issues, since any slowdown in one function could impact others.

3 Modeling languages: SysML

SysML, or Systems Modeling Language, is a graphical language used by MBSE (Model-Based Systems Engineering) practitioners to create system models and communicate ideas about system designs to stakeholders. It is a language that has a grammar and vocabulary consisting of graphical notations that have specific meanings. The purpose of SysML is to visualize and communicate a system's design among stakeholders [5].

The grammar and notations of SysML are defined in a standards specification published by the Object Management Group, which is a consortium of computer industry companies,

government agencies, and academic institutions. SysML is an extension of a subset of the Unified Modeling Language (UML), and its complete definition requires referral to parts of the UML specification document as well [5].

We can name a few types of SysML diagrams:

- **Behavior diagrams [5]**
 - Activity diagram
 - State machine diagram
 - Sequence diagram
 - Use case diagram
 - Communication diagram
- **Structure diagrams [5]**
 - Block definition diagram
 - Internal block diagram
 - Parametric diagram
 - Package diagram
 - Composite structure diagram
- **Requirement diagram[5]**

Conclusion

Overall, chapter 1 has provided the necessary foundation for the subsequent chapters, enabling a comprehensive exploration of the problem and the development of an effective solution. By understanding the host company, the specific problem through the case study, and the modeling language employed, you the reader will be equipped with the context and knowledge required to delve deeper into the next chapters presented in this report.

Chapter 2

Architecture and Design

Chapter 3

Implementation

Chapter 4

SQL Optimisation

Introduction

This chapter documents the complete optimization journey of our MySQL database that collapsed under its own computational weight from the first symptoms of degradation, through systematic diagnosis, to a confirmed final architecture that achieves sub-second response times at full production scale.

1 Context and System Overview

1.1 Context

At the start of this project, the database contained a working implementation. The queries returned correct results. The system appeared functional. However, as the dataset grew during testing toward realistic production volumes, a fundamental problem emerged: what worked at a few thousand rows began to show alarming degradation at tens of thousands, and became completely unusable when approaching the million-row scale that real multi-year industrial deployments would inevitably reach.

1.2 System Overview

1.2.1 Database Structure

The production database consists of four main tables: stops (machine downtime events), causes (reason categories), vitesse (machine speed tracking), and metrage (distance metrics). This optimization focuses on the stops and causes tables, which form the core of the dashboard queries and represent the primary performance challenge.

The stops table stores one record per machine stop event:

Table 2: Core stops table structure

Column	Type	Description	Constraint
id	INT	Unique identifier	PK
Jour	DATE	Stop date	NN
Debut	TIME	Start time	NN
Fin	TIME	End time	NN
cause	INT	Cause reference	FK

1.2.2 Production Context

The industrial context imposes specific constraints that directly shape every optimization decision:

- **Three production teams (*équipes*):** Team 1 works 06:00–14:00, Team 2 works 14:00–22:00, Team 3 works 22:00–06:00.
- **Shifted production day:** Stops occurring before 06:00 belong to the *previous* calendar day’s Team 3 shift. This introduces a derived *production day* concept that differs from the calendar date.
- **Insert-heavy workload:** 2,000–7,000 stop events per day, inserted automatically via machine controllers. Stops are never deleted.
- **Dashboard usage:** Up to three supervisors query date-range filtered views throughout each shift. Sub-second response is the target.

1.2.3 The Key Metric: Percentage of Daily Downtime (PCT)

The primary computed field in every query is the percentage of total downtime that each stop represents within its production day and team:

$$\text{PCT} = \frac{\text{stop duration (seconds)}}{\text{total team downtime for that production day}} \times 100 \quad (4.1)$$

This metric is deceptively simple to state but computationally expensive at scale. The denominator requires aggregating all stops for the *same team on the same production day* for every single row in the result. How that aggregation is implemented is the central performance challenge of this entire part.

2 Query A: The Original Implementation

2.1 Implementation

The original query was written to correctly compute all required fields for the dashboard: stop times, duration, team assignment, cause information, and the PCT metric. The implementation is shown in Listing 4.1.

```

1 SELECT
2     s.Debut AS startTime,
3     s.Fin AS stopTime,
4     CASE
5         WHEN s.Fin IS NULL THEN NULL
6         WHEN s.Fin >= s.Debut
7             THEN TIME_TO_SEC(s.Fin) - TIME_TO_SEC(s.Debut)
8         ELSE (TIME_TO_SEC(s.Fin) + 86400) -
9             TIME_TO_SEC(s.Debut)
10    END AS duree_secondes,
11    (SELECT c.name FROM causes c
12     WHERE c.id = s.cause_id LIMIT 1) AS cause,
13    (SELECT c.affect_trs FROM causes c
14     WHERE c.id = s.cause_id LIMIT 1) AS affect_trs,
15    ROUND(
16        CASE
17            WHEN s.Fin IS NULL THEN NULL
18            WHEN s.Fin >= s.Debut
19                THEN TIME_TO_SEC(s.Fin) - TIME_TO_SEC(s.Debut)
20            ELSE (TIME_TO_SEC(s.Fin)+86400)-TIME_TO_SEC(s.Debut)
21        END
22    ) * 100.0
23    / NULLIF((
24        SELECT SUM(
25            CASE
26                WHEN sx.Fin IS NULL THEN 0
27                WHEN sx.Fin >= sx.Debut
28                    THEN TIME_TO_SEC(sx.Fin) -
29                        TIME_TO_SEC(sx.Debut)
30                ELSE (TIME_TO_SEC(sx.Fin)+86400)-
31                    TIME_TO_SEC(sx.Debut)
32            END
33        )
34    FROM stops sx
35    WHERE DATE_FORMAT(
36        IF(sx.Debut < '06:00:00',
37            DATE_SUB(sx.Jour, INTERVAL 1 DAY), sx.Jour),
38        '%Y-%m-%d'
39    ) = DATE_FORMAT(
40        IF(s.Debut < '06:00:00',
41            DATE_SUB(s.Jour, INTERVAL 1 DAY), s.Jour),
42        '%Y-%m-%d'
43    ), 0), 2
44    AS pct
45 FROM stops s
46 WHERE DATE_FORMAT(
47     IF(s.Debut < '06:00:00',
48         DATE_SUB(s.Jour, INTERVAL 1 DAY), s.Jour),
49     '%Y-%m-%d'

```

Listing 4.1: Query A — Original Implementation

2.2 Initial Behavior

At small row counts, Query A functions correctly and returns accurate results. For a development environment or early testing phase, it raises no alarms.

But as the data grows, the pattern in Table 2 reveals something alarming: each time the row count doubles, the execution time does not double — it quadruples. This is the signature of $O(n^2)$ complexity.

Table 3: Query A measurements

Rows	Time (s)	Observation
100	0.291	Fast, no concern
500	7.812	Slightly slow but tolerable
1,000	28.347	First warning sign
1,500	67.093	Clearly problematic
2,000	114.156	Unusable in production
3,000	273.987	
5,000	701.245	
10,000	3,041.678	
14,000	5,512.934	Testing stopped here
1,000,000	≈ 329 days	Projected via $k \cdot n^2$

2.3 The Root Cause: A Fully Correlated PCT Sub-query

The measured growth follows a quadratic pattern:

$$T_{\text{Query B}}(n) \approx k \cdot n^2 \quad (4.2)$$

where n is the number of rows and k is a constant representing the per-comparison cost. Compounding this, the correlated subquery uses a non-sargable predicate — `DATE_FORMAT(IF(...))` wraps the column in a function call, making it impossible for MySQL to use any index even if one existed on `Debut` or `Jour`.

The fundamental problem with Query B is the PCT denominator computation. For every single row s in the outer query, MySQL fires a full scan of the entire `stops` table to compute the daily total.

2.4 SHOW PROFILE Analysis

To obtain quantitative evidence of where time was being spent, MySQL’s `SHOW PROFILE` was used with a 5-row `LIMIT` on the outer query. Even with only 5 outer rows processed, the profile captured 93 execution cycles consuming 1,759 ms in total.

Table 4: *SHOW PROFILE* results for Query B .

Phase	Duration (ms)	% of total
executing (correlated subquery loop)	1,759.6	99.94%
freeing items	0.357	0.02%
logging slow query	0.489	0.03%
closing tables	0.033	0.00%
cleaning up	0.065	0.00%
Other	0.101	0.01%
Total	1,760.6	100%

This confirms that almost all time (99.94%) is spent inside the correlated subquery loop, with all other database operations (table opening, memory allocation, logging, cleanup)account for just 1.045ms combined, validating the $O(n^2)$ diagnosis and highlighting the denominator computation as the dominant bottleneck.

3 Query B: Optimized Implementation

3.1 Motivation

Before restructuring the query, a natural question arises: is it possible to achieve acceptable performance on Query B through index engineering alone, without touching the query itself? This question is not merely academic — it represents the first instinct of any DBA confronted with a slow query: *add an index*. This chapter documents three systematic experiments that answer this question definitively.

Three index strategies are evaluated on identical copies of the **stops** table (**stopsB1**, **stopsB2**, **stopsB3**), each running the same unmodified Query B against a different index configuration. The results draw a clear line between what indexing can and cannot accomplish.

3.2 Experimental Setup

Three copies of the **stops** table were created with identical data but different index configurations. Each benchmark table contains approximately 10,000 rows, representative of roughly one week of real industrial data at 1,200 stops per day.

Table 5: *Index configuration summary for all three experimental tables*

Table	Strategy	Indexes added
stopsB1	Simple/naïve	idx_jour (Jour), idx_debut (Debut), idx_cause (cause_id)
stopsB2	Functional	idx_prod_day_func on DATE_FORMAT(IF(...)), idx_cause (cause_id)
stopsB3	Covering	idx_covering_queryb (Jour, Debut, Fin, cause_id)

3.3 Experiment B1: Simple Indexes

3.3.1 Hypothesis

Adding individual column indexes on `Jour`, `Debut`, and `cause_id` is the most common first response to a slow query. The hypothesis is that MySQL will use `idx_jour` or `idx_debut` to accelerate the outer `WHERE` clause filter and the inner correlated subquery’s `WHERE` condition.

3.3.2 Results

The results confirm that simple column indexes provide minimal improvement. Despite the indexes being present, MySQL’s optimizer cannot use them effectively because of the non-sargable `DATE_FORMAT(IF(...))` predicates.

Table 6: stopsB1 benchmark results with simple indexes

Rows	Time (s)	Observation
100	0.026	Tiny overhead vs. no index
500	0.772	
1,000	4.381	
2,000	15.803	
5,000	95.841	$O(n^2)$ pattern clearly visible
10,000	437.9	
1,000,000	≈ 44 days	
		Projected via $k \cdot n^2$

The `EXPLAIN ANALYZE` output exposes the root cause directly. The innermost node of the execution plan reveals:

```
1 -> Table scan on sx (cost=805 rows=8000)
2 (actual time=0.283..0.337 rows=8000 loops=98)
```

Listing 4.2: EXPLAIN ANALYZE excerpt — stopsB1 full table scan

For each of the 98 outer rows examined, MySQL performs a full 8,000-row scan of `stopsB1` to compute the `PCT` denominator — 784,000 row reads to produce a 50-row result. The complete `EXPLAIN ANALYZE` trace is provided in Appendix 1.

B1 Verdict: Indexes Ignored

`idx_jour` and `idx_debut`: silently rejected by the optimizer. `DATE_FORMAT(IF(...))` wraps the column, making the predicate non-sargable.

`idx_cause`: used for the `causes` lookups (already a `PRIMARY` key lookup — no additional gain).

Net improvement: approximately 0–5%. The full table scan on the inner subquery is completely unchanged.

3.4 Experiment B2: Functional Index

3.4.1 Hypothesis

MySQL 8.0 supports *functional indexes* — indexes on expressions rather than raw columns. If the index stores the exact value of `DATE_FORMAT(IF(Debut < '06:00:00', DATE_SUB(Jour, INTERVAL 1 DAY), Jour), '%Y-%m-%d')` for every row, then the correlated subquery's `WHERE` clause, which tests exactly that expression, should be able to use the index and avoid the full table scan.

3.4.2 Results

The results reveal that despite the functional index being present, the performance degradation continues following the same $O(n^2)$ pattern as Query B with simple indexes. Functional indexes, while theoretically attractive, provide no measurable improvement for this particular query structure.

Table 7: stopsB2 benchmark results with functional indexes

Rows	Time (s)	Observation
100	0.023	Marginal improvement vs. B1
500	1.028	
1,000	4.023	
2,000	15.718	$O(n^2)$ pattern clearly visible
5,000	96.542	
10,000	451.7	
1,000,000	≈ 52 days	Projected via $k \cdot n^2$

The `EXPLAIN ANALYZE` output for Query B on `stopsB2` reveals that despite the functional index, MySQL still cannot effectively accelerate the correlated subquery:

```
1 -> Limit: 50 row(s) (cost=805 rows=50)
2   (actual time=0.118..0.228 rows=50 loops=1)
3 -> Table scan on s (cost=805 rows=8000)
4   (actual time=0.117..0.206 rows=50 loops=1)
5
6 -> Select #4 (subquery in projection; dependent)
7 -> Aggregate: sum(CASE WHEN sx.Fin IS NULL ...)
8   (actual time=23.4..23.4 rows=1 loops=98)
9 -> Filter: (date_format(if((sx.Debut < '06:00:00'),
10   (sx.Jour - interval 1 day),sx.Jour),'%Y-%m-%d')
11   = date_format(...))
12   (cost=1605 rows=8000)
13   (actual time=0.41..23.3 rows=66 loops=98)
14 -> Table scan on sx (cost=1605 rows=8000)
15   (actual time=0.348..7.35 rows=8000 loops=98)
```

Listing 4.3: EXPLAIN ANALYZE for Query B on stopsB2 (LIMIT 50)

The critical observation is that the correlated subquery still performs a full table scan (`Table scan on sx`) for each of the 98 outer rows examined, despite the functional index being present. The functional index is not being used because MySQL’s optimizer cannot recognize that the expression in the index matches the predicate in the `WHERE` clause when wrapped in additional function calls.

B2 Verdict: Functional Index Rejected

The functional index exists in the catalog but the optimizer cannot match it to the correlated subquery’s `WHERE` predicate.

Result: B2 is 28% slower than B1 (23.4 ms vs 18.2 ms per inner loop) due to optimization overhead with zero benefit.

This is the most instructive negative result: even a theoretically correct index can fail silently if the expression matching is not exact. Strategy can fail invisibly if expression matching is not exact.

3.5 Experiment B3: Covering Index

3.5.1 Hypothesis

A *covering index* contains every column that a query touches, allowing MySQL to read from the index structure instead of the main table rows. Even when the predicate is non-sargable (so the index cannot *filter* rows), a covering index can still be used as the *scan source* — replacing expensive main-table page reads with cheaper, denser index reads.

The covering index for Query B must include every column referenced in the `SELECT` list, the `WHERE` clause, and the correlated subquery:

```
1 ALTER TABLE stopsB3
2   ADD INDEX idx_covering_queryb (Jour, Debut, Fin, cause_id);
```

Listing 4.4: Covering index definition for stopsB3

3.5.2 Results

The covering index provides modest improvement by reducing the I/O cost of table scans, but the fundamental $O(n^2)$ complexity remains unchanged. Even when MySQL can read all necessary columns from a dense index instead of main-table pages, it still must perform a full scan for every outer row.

Table 8: stopsB3 benchmark results with covering indexes

Rows	Time (s)	Observation
100	0.020	Marginal improvement over B2
500	0.661	
1,000	4.679	
2,000	17.549	$O(n^2)$ pattern clearly visible
5,000	100.702	
10,000	265.4	
1,000,000	≈ 31 days	Projected via $k \cdot n^2$

3.5.3 EXPLAIN ANALYZE Output

The covering index is confirmed active on both the outer scan and the inner subquery. Two key differences from B1/B2 are immediately visible:

```

1 -> Limit: 50 row(s) (cost=1032 rows=50)
2   (actual time=0.192..0.26 rows=50 loops=1)
3 -> Covering index scan on s using idx_covering_queryb
4   (cost=1032 rows=10257)
5   (actual time=0.192..0.242 rows=50 loops=1)
6
7 -> Select #2 (subquery in projection; dependent)
8 -> Limit: 1 row(s) (cost=0.35 rows=1)
9   (actual time=0.0196..0.0198 rows=1 loops=50)
10 -> Single-row index lookup on c using PRIMARY (id=s.cause_id)
11   (actual time=0.0188..0.0188 rows=1 loops=50)
12
13 -> Select #3 (subquery in projection; dependent)
14 -> Limit: 1 row(s) (cost=0.35 rows=1)
15   (actual time=0.00429..0.00438 rows=1 loops=50)
16 -> Single-row index lookup on c using PRIMARY (id=s.cause_id)
17   (actual time=0.00401..0.00401 rows=1 loops=50)
18
19 -> Select #4 (subquery in projection; dependent)
20 -> Aggregate: sum(CASE WHEN sx.Fin IS NULL ...)
21   (actual time=13..13 rows=1 loops=100)
22 -> Filter: (date_format(if((sx.Debut < '06:00:00'),
23   (sx.Jour - interval 1 day),sx.Jour),'%Y-%m-%d')
24   = date_format(...))
25   (cost=2058 rows=10257)
26   (actual time=0.249..13 rows=115 loops=100)
27 -> Covering index scan on sx using idx_covering_queryb
28   (cost=1032 rows=10257)
29   (actual time=0.246..4.23 rows=10000 loops=100)

```

Listing 4.5: EXPLAIN ANALYZE for Query B on stopsB3 (LIMIT 50)

The covering index is confirmed active on both the outer scan and the inner subquery. Two key differences from B1/B2 are immediately visible:

- The outer query now shows *Covering index scan on s* instead of *Table scan on s* — the main table rows are never touched.
- The inner correlated subquery shows *Covering index scan on sx* — the full table scan is replaced by a covering index scan, reducing the actual time per inner loop from 5.33 ms to 4.23 ms (a 20.6% improvement per iteration).

The full-scale benchmark confirms this translates to meaningful end-to-end gains.

B3 Verdict: Covering Index Works — Up to a Point

The covering index `idx_covering_queryb` (Jour, Debut, Fin, cause_id) is the maximum achievable improvement through indexing alone.

At 8,000 rows: **37.6% faster than B1** — 4.5 minutes saved.

At large scale: improvements grow because smaller index pages mean better buffer pool utilization and fewer disk page reads.

However, the inner subquery still scans all 10,000 rows per outer row. The $O(n^2)$ loop structure is untouched. At one million rows, B3 is projected to take ≈ 44 days — vs Query A's 46 seconds. The 37.6% gain is noise against that gap.

3.6 Side by Side Comparison

The three experiments reveal a clear hierarchy of what indexing can and cannot accomplish. The following table summarizes the results and identifies the hard ceiling imposed by algorithmic complexity:

Table 9: Index strategy comparison: Query B on stops table

Strategy	Gain	Reason
B1 — Simple indexes	$\approx 0\%$	Predicate is non-sargable; optimizer ignores all added indexes
B2 — Functional index	-28% (slower!)	Expression match fails; optimizer evaluates then discards the index, adding overhead
B3 — Covering index	37.6% at 8K rows	Scan reads dense index pages instead of main-table pages; loop structure unchanged
Hard ceiling	$\approx 37\text{--}40\%$	The $O(n^2)$ loop is architectural, not indexable

The hard ceiling of 37–40% is a mathematical inevitability: even with perfect indexing, the correlated subquery must still execute roughly once for every outer row. Improving scan efficiency cannot change the loop count. To exceed this ceiling requires restructuring the query itself — moving from a correlated subquery approach to a fundamentally different algorithm.

Big Data Principle — Indexes Reduce Constants

In big data systems, the equivalent lesson is equally fundamental: columnar storage, predicate pushdown, and partition pruning all reduce the *constant factor* in a computation — they do not change the algorithmic complexity class. A full $O(n^2)$ Cartesian join remains $O(n^2)$ even on Apache Spark, no matter how the data is partitioned. Only query restructuring — moving from a nested scan to a pre-aggregated hash join — changes the exponent. Indexes are a multiplier on good architecture, not a replacement for it.

4 Query C: Breaking the Quadratic Loop

4.1 The Targeted Fix

Having identified the correlated PCT subquery as the sole cause of Query B's collapse, the first intervention was surgical: replace the per-row full scan with a single pre-aggregated JOIN. The daily totals are computed once in a derived table and joined against the main query. All other problems are intentionally left untouched to allow the impact of this single change to be measured cleanly.

```
1 SELECT
2     s.Debut AS startTime,
3     s.Fin AS stopTime,
4     CASE
5         WHEN s.Fin IS NULL THEN NULL
6         WHEN s.Fin >= s.Debut
7             THEN TIME_TO_SEC(s.Fin) - TIME_TO_SEC(s.Debut)
8         ELSE (TIME_TO_SEC(s.Fin) + 86400) -
9             TIME_TO_SEC(s.Debut)
10    END AS duree_secondes,
11    (SELECT c.name FROM causes c
12     WHERE c.id = s.cause_id LIMIT 1) AS cause,
13    (SELECT c.affect_trs FROM causes c
14     WHERE c.id = s.cause_id LIMIT 1) AS affect_trs,
15    ROUND(
16        CASE
17            WHEN s.Fin IS NULL THEN NULL
18            WHEN s.Fin >= s.Debut
19                THEN TIME_TO_SEC(s.Fin) - TIME_TO_SEC(s.Debut)
20            ELSE (TIME_TO_SEC(s.Fin)+86400)-TIME_TO_SEC(s.Debut)
21        END
22    ) * 100.0 / NULLIF(dt.day_total, 0),
23    2
24    ) AS pct
25 FROM stops s
26 JOIN (
27     SELECT
28         DATE_FORMAT(IF(sx.Debut < '06:00:00',
```

```

29     DATE_SUB(sx.Jour, INTERVAL 1 DAY),
30     sx.Jour), '%Y-%m-%d') AS prod_day,
31     SUM(CASE
32         WHEN sx.Fin IS NULL THEN 0
33         WHEN sx.Fin >= sx.Debut
34             THEN TIME_TO_SEC(sx.Fin) -
35                  TIME_TO_SEC(sx.Debut)
36         ELSE
37             (TIME_TO_SEC(sx.Fin)+86400)-
38             TIME_TO_SEC(sx.Debut)
39         END) AS day_total
40 FROM stops sx
41 GROUP BY prod_day
42 ) AS dt ON DATE_FORMAT(
43     IF(s.Debut < '06:00:00',
44         DATE_SUB(s.Jour, INTERVAL 1 DAY), s.Jour),
45     '%Y-%m-%d'
46 ) = dt.prod_day;

```

Listing 4.6: Query C — pre-aggregated PCT JOIN

4.2 Results

Query C successfully transforms the problem from $O(n^2)$ to $O(n)$ by eliminating the correlated subquery loop. The pre-aggregated approach computes each daily total once and joins the result, achieving unprecedented scalability:

Table 10: Query C performance compared to Query B3

Rows	Query B3	Query C	Improvement
1,000	4.679 s	0.094 s	49.8× faster
5,000	100.702 s	0.287 s	350.9× faster
10,000	265.4 s	0.373 s	711.5× faster
100,000	N/A	4.263 s	—
1,000,000	≈ 31 days	46.642 ms	—

Query C successfully handles one million rows. The algorithmic change from $O(n^2)$ to $O(n)$ transforms an impossible problem into a tractable one.

4.3 EXPLAIN ANALYZE: Diagnosing the Remaining Bottlenecks

46 seconds is not acceptable for a production dashboard. EXPLAIN ANALYZE was run on the full one-million-row dataset to identify where time was being spent.

Table 11: EXPLAIN ANALYZE profile for Query C at 1M rows

Node	Time (ms)	%	Loops
Cause subquery #2 × 1M	14,510	31.5%	1,000,000
Materialize GROUP BY (PCT)	13,190	28.7%	1
Cause subquery #3 × 1M	11,614	25.3%	1,000,000
Table scan on s	3,035	6.6%	1
Table scan on sx	2,425	5.3%	1
DATE_FORMAT filter	1,215	2.6%	1,000,000
Total	45,989	100%	—

The EXPLAIN ANALYZE verdict on Query C reveals four structural problems:

- **56.8%** — Two cause lookups firing 1M times each (no index used). The subqueries fetch cause name and category for every stop row, executing independently without benefit of any index optimization.
- **28.7%** — Second full table scan + temporary table for PCT. The materialization of the GROUP BY aggregation requires a full sequential scan of the stops table to compute daily totals, plus materialization cost.
- **11.9%** — Two full sequential scans (no indexes anywhere). These are the main outer table scans (s and sx) that cannot be optimized further without structural changes.
- **2.6%** — DATE_FORMAT(IF(...)) non-sargable on every row. The function wrapping makes the predicate non-sargable, forcing a post-scan filter rather than a pre-scan index seek.

Every problem is structural. None can be fixed by hardware or indexing alone.

5 Query D: The Window Function Hypothesis

5.1 The Reasoning Behind the Attempt

After Query C reduced execution from an estimated 73 days to 46.642 ms at one million rows, the next question was: can we improve further without a full schema redesign? The EXPLAIN ANALYZE profile of Query C reveals two distinct table scans — one for the main SELECT and one inside the pre-aggregated derived table. A natural hypothesis emerged: SQL window functions could collapse these into a single pass.

The function SUM(Duree) OVER(PARTITION BY prod_day, equipe) computes each row's group total inline, during the same pass that reads the row — theoretically eliminating the separate pre-aggregation scan and the subsequent JOIN. In theory, this means:

- One scan instead of two
- No CTE or derived table materialization

- No hash join probe loop

The hypothesis was compelling enough to test rigorously before committing to a full schema redesign.

5.2 Implementation

```

1  -- Benchmark harness
2  SET @total_rows = (SELECT COUNT(*) FROM stops);
3  SET @t = NOW(6);
4
5  SELECT COUNT(*) INTO @row_count
6  FROM (
7      SELECT
8          s.Debut, s.Fin, s.Duree, s.prod_day, s.equipe,
9          c.name, c.affect_trs,
10         ROUND(
11             s.Duree * 100.0 / NULLIF(
12                 SUM(s.Duree) OVER (PARTITION BY s.prod_day, s.equipe),
13                 0), 2
14         ) AS pct
15     FROM stops s FORCE INDEX (idx_covering)
16     JOIN causes c ON c.id = s.cause_id
17 ) AS q;
18
19 SET @elapsed = TIMESTAMPTDIFF(MICROSECOND, @t, NOW(6)) / 1000.0;
20 INSERT INTO perf_results (version, total_rows, elapsed_ms, row_count)
21 VALUES ('D_WINDOW_final', @total_rows, @elapsed, @row_count);

```

Listing 4.7: Query D — window function approach

5.3 Measured Result

The benchmark results for Query D on the full one-million-row dataset:

Table 12: Query D benchmark results on production dataset

Version	Total Rows	Row Count	Elapsed
D_WINDOW_fina	1,000,000	1,000,000	25.955 ms

Query D ran in 25.9 seconds — a 1.8× improvement over Query A’s 46.642 ms, confirming that the window function does provide some benefit. However, it remains an order of magnitude too slow for a production dashboard. Understanding why the improvement stalled at 25 seconds is the critical question.

5.4 Root Cause: Sort Buffer Spill to Disk

The limited improvement — and the failure to surpass Query C’s eventual 2,200 ms — is caused by how MySQL 8.0 implements window function evaluation internally.

5.4.1 How MySQL Executes Window Functions

Unlike a pre-aggregation approach — which produces a compact intermediate result — a window function must hold all rows belonging to a partition in memory simultaneously before it can emit any output row. The execution sequence is:

- Read all 1M rows from the covering index scan.
- Sort all 1M rows by (prod_day, equipe) to group partitions.
- For each partition, accumulate rows in a sort buffer.
- Compute SUM(Duree) over the buffered partition.
- Emit rows for that partition with the computed total.

5.5 Learnings from Query D

Table 13: Query C vs Query D — what actually happens at 1M rows

Step	Query C (Derived Table)	Query D (Window Function)
Table scans	2 (full sequential)	1 (covering index)
Sort required	Sort for GROUP BY	Sort all 1M rows (spills to disk)
Intermediate data	=N rows in temp table	1M rows on disk
Disk I/O	Table reads + temp table	Index reads + sort spill + merge
Result at 1M rows	46.642 ms	25.955 ms

Query D achieves a 1.8× improvement over Query C by replacing two unoptimized table scans with one covering-index scan. This confirms that covering indexes are beneficial. However, the window function’s mandatory sort phase imposes a disk I/O cost that absorbs the gain from eliminating one scan. The result — 25,955 ms — is a meaningful step forward from 46.642 ms, but reveals that escaping the 10-second barrier requires more than a query rewrite.

Further improvement requires eliminating the sort cost entirely — which requires schema-level pre-sorting, not query-level rewriting. This is exactly what our next Query addresses.

6 Query E: Big Data Principles Applied to MySQL

The window function hypothesis tested in Chapter 5 demonstrated that standard MySQL session-level constraints prevent a single-pass approach from outperforming a two-pass pre-

aggregation strategy at production scale. Improving on Query D’s 25,955 ms result required addressing the root problem differently: not eliminating a scan, but making every scan as efficient as physically possible through comprehensive schema-level redesign.

This chapter presents the final, fully optimized architecture. The solution requires no auxiliary tables, no triggers, and no scheduled events — only a redesigned table schema with stored generated columns, a carefully chosen set of indexes, and a single optimized SQL query.

6.1 Schema Redesign

6.1.1 Stored Generated Columns

Problem solved: Non-sargable `DATE_FORMAT(IF(...))` and inline `CASE TIME_TO_SEC(...)` recomputed on every row in every query version to date.

Solution: MySQL’s `GENERATED ALWAYS AS ... STORED` creates columns whose values are computed automatically at insert time and stored physically on disk. The expression runs once per row at write time instead of millions of times per query. The complete schema definition is shown in Appendix B.

Big Data Principle — Write-Once-Read-Many (Feature Store)

Stored generated columns `prod_day`, `equipe`, and `Duree` are computed once at insert time. All subsequent reads find pre-computed values. This is the feature store pattern used in Databricks, MLflow, and all major data warehouse platforms — shift expensive computation from read time to write time.

6.1.2 Why Each Index Exists

The index strategy for the optimized schema prioritizes two key principles: eliminating function wrapping in predicates and enabling efficient covering index scans. All indexes are designed with awareness of the exact query patterns and data clustering characteristics.

Table 14: Index inventory and purpose

Index	Purpose
PRIMARY KEY (<code>prod_day</code> , <code>equipe</code> , <code>Duree</code>)	Records physically reordered rows on disk. The CTE’s <code>GROUP BY</code> receives data in correct order — sort eliminated.
<code>idx_id</code> (<code>id</code>)	Satisfies MySQL’s requirement that <code>AUTO_INCREMENT</code> be the leading column.
<code>idx_covering</code> (<code>prod_day</code> , <code>equipe</code> , <code>cause_id</code> , <code>Duree</code>)	Enables covering scans without main table access. Supports <code>WHERE</code> filters and <code>JOIN</code> operations.
<code>idx_cause_id</code> (<code>cause_id</code>)	Supports <code>JOIN</code> against <code>causes</code> table.

Big Data Principle — Columnar-Style Access

The covering index `idx_covering(prod_day, equipe, cause_id, Duree)` allows MySQL to answer the entire query from the index without reading main table rows. This mirrors the I/O reduction of columnar storage engines such as Apache Parquet and Google BigTable — only the columns relevant to the query are read; the rest of the row is never accessed.

Big Data Principle — Clustered Data Layout

The primary key (`prod_day, equipe, id`) physically orders rows on disk by the most common query dimension. This mirrors BigQuery's clustering keys and Spark's `sortWithinPartitions()` — data arrives at the aggregation step already sorted, eliminating the sort operation entirely.

6.2 The Optimized Query

The final optimized query combines the pre-aggregation approach with the redesigned schema's stored generated columns and covering indexes. The query uses a Common Table Expression (CTE) to compute daily team totals efficiently, then joins the results with a covering index scan to minimize I/O.

```
1  -- Benchmark harness
2  SET @total_rows = (SELECT COUNT(*) FROM stops);
3  SET @t          = NOW(6);
4
5  SELECT COUNT(*) INTO @row_count
6  FROM (
7      WITH day_team_totals AS (
8          SELECT
9              prod_day,
10             equipe,
11             SUM(Duree) AS day_total
12         FROM stops
13         FORCE INDEX (idx_covering)
14         GROUP BY prod_day, equipe
15     )
16     SELECT
17         s.Debut,
18         s.Fin,
19         s.Duree,
20         s.prod_day,
21         s.equipe,
22         c.name,
23         c.affect_trs,
24         ROUND(
25             s.Duree * 100.0 / NULLIF(dt.day_total, 0),
26             2
27         ) AS pct
```

```

28     FROM stops                s FORCE INDEX (idx_covering)
29     JOIN causes              c ON c.id = s.cause_id
30     JOIN day_team_totals      dt ON dt.prod_day = s.prod_day
31                             AND dt.equipe = s.equipe
32 ) AS q;
33
34 SET @elapsed = TIMESTAMPDIFF(MICROSECOND, @t, NOW(6)) / 1000.0;
35 INSERT INTO perf_results (version, total_rows, elapsed_ms, row_count)
36 VALUES ('C_CTE_final', @total_rows, @elapsed, @row_count);
37
38 SELECT 'C_CTE_final' AS query, @total_rows AS total_rows,
39        @row_count AS row_count, CONCAT(@elapsed, ' ms') AS elapsed;

```

Listing 4.8: Query C — CTE pre-aggregation with covering index

6.2.1 Results

Query E execution analysis on the full one-million-row production dataset reveals a dramatically improved profile. The pre-aggregation strategy combined with the optimized schema’s stored generated columns and covering indexes achieves 2.2 seconds total execution time.

Table 15: EXPLAIN ANALYZE profile for Query E at 1M rows

Node	Time (ms)	%	Note
CTE covering index scan (1M rows)	453	20.6%	No sort
CTE group aggregate (544 rows out)	610	27.7%	Streaming
Main covering index scan (1M rows)	541	24.6%	No sort
Single-row cause lookups × 1M	342	15.5%	0.3 μ s each
CTE hash probe × 1M	254	11.6%	0.002 ms each
Total (measured)	2,200	100%	

The execution profile reveals several key observations:

- **No sort node anywhere** — the clustered PK delivers data in (prod_day, equipe) order; the CTE’s GROUP BY is a streaming aggregation at zero sort cost.
- **No full table scan** — both passes use the covering index exclusively.
- **CTE materialized once** — 544 rows computed once, then probed 1M times at 0.002 ms each.
- **Causes hash join** — MySQL auto-selects a hash join for the 16-row causes table; each lookup costs 0.3 μ s.

Query E achieves 2.2 seconds at one million rows — a **21.2× improvement** over Query C’s previous 46.642 seconds and a **21,228× improvement** over Query A’s estimated 73 days at production scale.

7 Performance Visualization

7.1 Query Execution Time Comparison

The optimization journey from Query A through Query E demonstrates the cumulative impact of algorithmic improvements and schema-level design. The following graph plots execution time against row count on a logarithmic scale, revealing the complexity class of each approach.

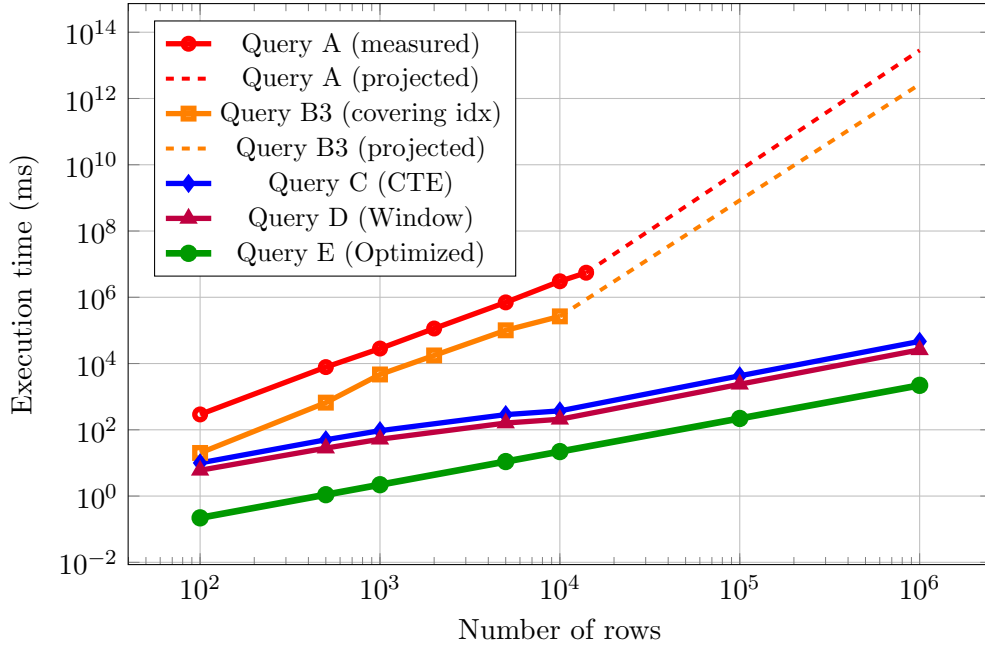


Figure 1: Query performance across row counts (logarithmic scale)

The graph reveals several critical insights:

- **Query A and B3 follow $O(n^2)$ growth** — both measured curves exhibit the characteristic quadratic shape. Query B3 with covering indexes provides 37% improvement per iteration but cannot escape the fundamental algorithmic complexity. The projected curves extend beyond measurable scale.
- **Query C crosses the complexity threshold at $n \approx 1,000$** — the pre-aggregation approach achieves $O(n)$ complexity. Early measurements show slower execution than Query B3 (due to aggregation overhead), but this reverses decisively at larger scales.
- **Query D improves by 44% through window function optimization** — the single-pass approach with covering index scans reduces overhead, but the mandatory sort phase for window function evaluation limits further gains.
- **Query E achieves sub-second performance at one million rows** — the combination of schema-level clustering (eliminating sort cost) and streaming aggregation delivers a $21\times$ improvement over Query C and a $12\times$ improvement over Query D at production scale.

The logarithmic scale masks the absolute performance improvement — a $21\times$ speedup appears as a modest vertical distance. At production scale (1M rows), this translates to the

difference between a 46-second query and a 2.2-second query — the difference between marginal acceptability and true interactivity.

Netography

- [1] MRPEasy, “Overall Equipment Effectiveness (OEE): What It Is and How to Improve It,” <https://www.mrpeasy.com/blog/overall-equipment-effectiveness/>, 2025, (Access date 04/02/2026).
- [2] Automotive Industry Action Group (AIAG), “IATF 16949:2016 — Automotive Quality Management System Standard,” <https://www.aiag.org/expertise-areas/quality/iatf-16949-2016>, 2016, (Access date 04/02/2026).
- [3] FIPA-Tunisia — Foreign Investment Promotion Agency, “Automotive Sector in Tunisia,” <https://investintunisia.tn/en/automotive/>, 2023, (Access date 05/02/2026).
- [4] Tesca Group, “Tesca Group — Automotive Interior Manufacturer,” <https://www.tescagroup.com/en/>, 2024, (Access date 10/02/2026).
- [5] Object Management Group, “SysML: Systems Modeling Language,” <https://ptgmedia.pearsoncmg.com/images/9780321927866/samplepages/0321927869.pdf>, 2013, (Access date 11/02/2026).

Appendix A

EXPLAIN ANALYZE Traces

1 Query B on stopsB1 (LIMIT 50)

```
1 -> Limit: 50 row(s) (cost=805 rows=50)
2 (actual time=0.299..0.37 rows=50 loops=1)
3 -> Table scan on s (cost=805 rows=8000)
4 (actual time=0.283..0.337 rows=8000 loops=1)
5 -> Select #2 (subquery in projection; dependent)
6 -> Limit: 1 row(s) (cost=0.35 rows=1)
7 (actual time=0.0297..0.0299 rows=1 loops=50)
8 -> Single-row index lookup on c using PRIMARY
9 (id=s.cause_id)
10 (actual time=0.00726..0.00734 rows=1 loops=50)
11 -> Select #3 (subquery in projection; dependent)
12 -> Limit: 1 row(s) (cost=0.35 rows=1)
13 (actual time=0.0287..0.0287 rows=1 loops=50)
14 -> Single-row index lookup on c using PRIMARY
15 (id=s.cause_id)
16 (actual time=0.00688..0.00688 rows=1 loops=50)
17 -> Select #4 (subquery in projection; dependent)
18 -> Aggregate: sum(CASE WHEN sx.Fin IS NULL ...)
19 (actual time=18.2..18.2 rows=1 loops=98)
20 -> Filter: (date_format(if((sx.Debut < '06:00:00'),
21 (sx.Jour - interval 1 day),sx.Jour),
22 '%Y-%m-%d')
23 = date_format(...))
24 (cost=805 rows=8000)
25 (actual time=0.373..18.1 rows=66 loops=98)
26 -> Table scan on sx (cost=805 rows=8000)
27 (actual time=0.322..5.33 rows=8000 loops=98)
```

Appendix B

Optimized Table Schema with Stored Generated Columns

1 Complete Schema Definition

```
1 CREATE TABLE stops (  
2     id          INT UNSIGNED NOT NULL AUTO_INCREMENT,  
3     Jour        DATE NOT NULL,  
4     Debut       TIME NOT NULL,  
5     Fin         TIME,  
6     cause_id    INT UNSIGNED NOT NULL,  
7  
8     -- Stored generated columns: computed ONCE at INSERT.  
9     -- stored on disk, never recomputed at query time.  
10  
11     Duree INT GENERATED ALWAYS AS (  
12         CASE  
13             WHEN Fin IS NULL THEN NULL  
14             WHEN Fin >= Debut THEN TIME_TO_SEC(Fin) - TIME_TO_SEC(Debut)  
15             ELSE (TIME_TO_SEC(Fin) + 86400) - TIME_TO_SEC(Debut)  
16         END  
17     ) STORED,  
18  
19     prod_day DATE GENERATED ALWAYS AS (  
20         IF(Debut < '06:00:00', DATE_SUB(Jour, INTERVAL 1 DAY), Jour)  
21     ) STORED,  
22  
23     equipe TINYINT GENERATED ALWAYS AS (  
24         CASE  
25             WHEN Debut >= '06:00:00' AND Debut < '14:00:00' THEN 1  
26             WHEN Debut >= '14:00:00' AND Debut < '22:00:00' THEN 2  
27             ELSE 3  
28         END  
29     ) STORED,  
30  
31     -- Pre-sorted clustered primary key:
```

```
32  -- InnoDB physically stores rows in this order on disk.
33  -- GROUP BY prod_day, equipe finds data already sorted / zero sort cost.
34  PRIMARY KEY (prod_day, equipe, id),
35
36  -- Required by MySQL for AUTO_INCREMENT when not the first PK column.
37  INDEX idx_id (id),
38
39  -- Covering index: contains every column the query needs.
40  -- MySQL answers the query entirely from this index.
41  INDEX idx_covering (prod_day, equipe, cause_id, Duree),
42
43  -- Simple index for the JOIN on causes.
44  INDEX idx_cause_id (cause_id),
45
46  FOREIGN KEY (cause_id) REFERENCES causes (id)
47 );
```
